

Files

```
{x}
├── ..
├── logs
├── sample_data
├── logs.csv
├── model.keras
├── model.weights.h5
```

<>

Saved successfully!

```
[1] #How to install libraries that are not pre-installed on Colab
45 !pip install patchify
!pip install tensorflow-gpu

Collecting patchify
  Downloading patchify-0.2.3-py3-none-any.whl.metadata (3.0 kB)
Requirement already satisfied: numpy<2, >=1 in /usr/local/lib/python3.10/dist-packages (from patchify) (1.26.4)
  Downloading patchify-0.2.3-py3-none-any.whl (6.6 kB)
Installing collected packages: patchify
Successfully installed patchify-0.2.3
Collecting tensorflow-gpu
  Downloading tensorflow-gpu-2.12.0.tar.gz (2.6 kB)
error: subprocess-exited-with-error

  × python setup.py egg_info did not run successfully.
  │ exit code: 1
  └─> See above for output.

note: This error originates from a subprocess, and is likely not a problem with pip.
  Preparing metadata (setup.py) ... error
error: metadata-generation-failed

  × Encountered error while generating package metadata.
  └─> See above for output.

note: This is an issue with the package mentioned above, not pip.
hint: See above for details.
```

```
ds [2] #Import needed libraries
import tensorflow as tf
from tensorflow.keras import datasets
import numpy as np
import matplotlib.pyplot as plt
```

```
[3] #Check if GPU is used
my_gpu_device_name = tf.test.gpu_device_name()

if my_gpu_device_name != '/device:GPU:0':
    raise SystemError('GPU not found!')

print(f'Available GPU : {my_gpu_device_name}')
```

↔ Available GPU : /device:GPU:0

```
[4] #Check which GPU
gpu_info = !nvidia-smi
gpu_info = '\n'.join(gpu_info)

if gpu_info.find('failed') >= 0:
    print('Select the Runtime > "Change runtime type" menu to enable a GPU accelerator, ')
    print('and then re-execute this cell.')
else:
    print(gpu_info)
```

Wed Dec 18 07:43:13 2024

NVIDIA-SMI 535.104.05			Driver Version: 535.104.05			CUDA Version: 12.2		
GPU Name			Persistence-H		Bus-Id	Disp-A	Volatile Uncorr.	ECC
Fan Temp	Perf		Pwr:Usage/Cap			Memory-Usage	GPU-Util	Compute H
							MIG M	
0 Tesla T4			Off		00000000:00:04.0 Off		0	
N/A	37C	P0	27W / 70W		103MiB / 15360MiB		0%	Default
							N/A	

```

=====
Processes:
GPU   GI   CI           PID  Type   Process name          GPU Memory
  ID   ID                               Usage
=====

```

```
[5] #Check how much memory is still available
from psutil import virtual_memory
ram_gb = virtual_memory().total / 1e9
print('Your runtime has {:.1f} gigabytes of available RAM\n'.format(ram_gb))

if ram_gb < 20:
    print('To enable a high-RAM runtime, select the Runtime > "Change runtime type"'
          ' menu, and then select high-RAM in the Runtime shape dropdown. Then, ')
    print('re-execute this cell.')
else:
    _print('You are using a high-RAM runtime!')
```

→ Your runtime has 13.6 gigabytes of available RAM

To enable a high-RAM runtime, select the Runtime > "Change runtime type" menu, and then select High-RAM in the Runtime shape dropdown. Then, re-execute this cell.

```
[6] print(tf. version ) #show the version of TensorFlow we are using
```

2.17.1

```
[7] #Load the MNIST dataset
mnist = datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 0s 0us/step

```

```
19 [8]
#Visualise a few examples
plt.figure(figsize=(10, 2))
for index, (image, label) in enumerate(zip(x_train[0:5], y_train[0:5])):
    plt.subplot(1, 5, index + 1)
    print(np.shape(image))
    plt.imshow(image, cmap=plt.cm.gray)
    plt.title('Training: %3s'%label, fontsize = 10)
```

```
✓ [9] #Convert data from integers to floating-point numbers (between 0 and 1)
```

```
x_train, x_test = x_train / 255.0, x_test / 255.0

[10] #Build the tf.keras.Sequential model by adding a layer after another
model = tf.keras.models.Sequential()
model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))
model.add(tf.keras.layers.Dense(128, activation='relu'))
model.add(tf.keras.layers.Dropout(0.2))
model.add(tf.keras.layers.Dense(10, activation='softmax'))

/usr/local/lib/python3.10/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer us
super().__init__(**kwargs)

[11] #Compile the model : add optimizer, loss, and performance evaluation metrics
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              # 'sparse_categorical_crossentropy' used when the true labels are integers
              # and 'categorical_crossentropy' is used when the true labels are one-hot encoded
              metrics=['accuracy'])

[12] print(np.shape(x_train))
print(np.shape(x_test))

(60000, 28, 28)
(10000, 28, 28)

[13] #Train
model.fit(x_train, y_train, epochs=10) #default batch_size = 32

Epoch 1/10
1875/1875 ----- 6s 2ms/step - accuracy: 0.8753 - loss: 0.4438
Epoch 2/10
1875/1875 ----- 4s 2ms/step - accuracy: 0.9666 - loss: 0.1172
Epoch 3/10
1875/1875 ----- 3s 2ms/step - accuracy: 0.9769 - loss: 0.0761
Epoch 4/10
1875/1875 ----- 3s 2ms/step - accuracy: 0.9837 - loss: 0.0533
Epoch 5/10
1875/1875 ----- 4s 2ms/step - accuracy: 0.9873 - loss: 0.0407
Epoch 6/10
1875/1875 ----- 3s 2ms/step - accuracy: 0.9898 - loss: 0.0337
Epoch 7/10
1875/1875 ----- 3s 1ms/step - accuracy: 0.9921 - loss: 0.0263
Epoch 8/10
1875/1875 ----- 5s 2ms/step - accuracy: 0.9942 - loss: 0.0192
Epoch 9/10
1875/1875 ----- 4s 2ms/step - accuracy: 0.9957 - loss: 0.0156
Epoch 10/10
1875/1875 ----- 4s 2ms/step - accuracy: 0.9957 - loss: 0.0137
<keras.src.callbacks.history.History at 0x788f55effd60>

[14] #Evaluate
loss, accuracy = model.evaluate(x_test, y_test)
print("loss = {}, accuracy = {}".format(loss, accuracy))

313/313 ----- 1s 3ms/step - accuracy: 0.9782 - loss: 0.0804
loss = 0.07181509584188461, accuracy = 0.9807000160217285

[15] from sklearn import metrics
predictions = model.predict(x_test) #gives the probabilities of belonging of this image to the classes
print(predictions)

313/313 ----- 1s 2ms/step
[[2.3535883e-08 1.4502021e-09 1.3903417e-07 ... 9.9995756e-01
 1.6455604e-07 1.2154999e-06]
 [9.5516317e-11 1.6939266e-07 9.9999888e-01 ... 2.4590882e-17
 1.2407624e-09 4.9093331e-20]
 [9.5793820e-10 9.9971145e-01 6.9414607e-05 ... 1.4266891e-04
 6.4883316e-05 9.5192327e-08]
 ...
 [7.5017407e-10 7.7632772e-14 6.2735363e-18 ... 7.7881985e-09
 6.3607621e-09 1.7477227e-07]
 [1.2411262e-13 1.0422749e-12 1.5242888e-14 ... 3.4831491e-13
 2.3328674e-05 5.5313378e-15]
 [3.5886509e-12 1.3645600e-13 9.0018080e-12 ... 1.1216737e-16
 1.1391529e-12 8.6509450e-16]]

[16] #To get the classes:
classes = np.argmax(predictions, axis = 1) #finds the index of the highest value in each row, axis=1 means that the operation is across rows
print(classes)

[7 2 1 ... 4 5 6]

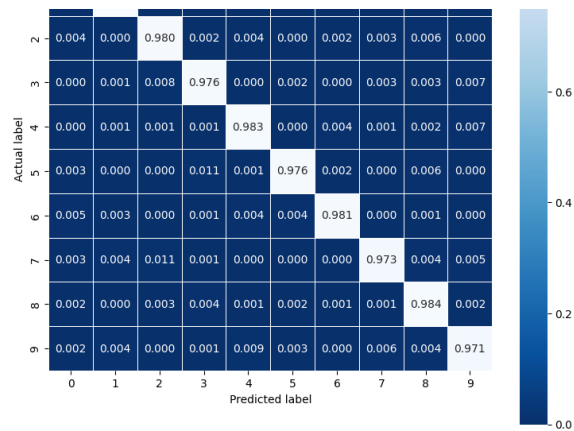
[17] #Visualise
plt.figure(figsize=(20, 4))
for index, (image, label) in enumerate(zip(x_test[0:10], classes[0:10])):
    plt.subplot(2, 5, index + 1)
    plt.imshow(image, cmap=plt.cm.gray)
    plt.title('Training: {}'.format(label), fontsize = 10)

Training: 7 Training: 2 Training: 1 Training: 0 Training: 4
0 0 0 0 0
10 10 10 10 10
20 20 20 20 20
0 10 20 0 10 20 0 10 20 0 10 20 0 10 20
0 10 20 0 10 20 0 10 20 0 10 20 0 10 20
0 10 20 0 10 20 0 10 20 0 10 20 0 10 20

[18] import seaborn as sns
cm = metrics.confusion_matrix(y_test, classes, normalize='true')
plt.figure(figsize=(9,9))
sns.heatmap(cm, annot=True, fmt=".3f", linewidths=.5, square = True, cmap = 'Blues_r')
plt.ylabel('Actual label')
plt.xlabel('Predicted label')
all_sample_title = 'Accuracy = {}'.format(accuracy)
plt.title(all_sample_title, size = 15)

Text(0.5, 1.0, 'Accuracy = 0.9807000160217285')

Accuracy = 0.9807000160217285
0 - 0.989 0.001 0.001 0.001 0.001 0.000 0.002 0.000 0.003 0.002
0.000 0.993 0.004 0.000 0.000 0.000 0.000 0.000 0.004 0.000
- 0.8
```



```
[27] #Callbacks : A callback performs actions at various stages of training (e.g. at the start or end of an epoch, before or after a single batch)
#Early Stopping
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, TensorBoard, CSVLogger
import os, datetime

early_stopping = EarlyStopping(monitor='val_loss', patience=5) #Which quantity to monitor to decide early stopping
#patience = the number of epochs to wait before stopping

#Notice that since we set patience=5, we won't get the best model, but the model 5 epochs after the best model.
#==> An additional callback is required that saves the best model observed during training. This is the ModelCheckpoint callback

model_checkpoint = ModelCheckpoint(filepath='model.keras', monitor='val_loss', save_weights_only=False, save_best_only=True)

logdir = os.path.join("logs", datetime.datetime.now().strftime("%Y%m%d-%H%M%S"))
tnsrboard = TensorBoard(logdir, histogram_freq=1)

csv_log_path = "/content/logs.csv"
csv_logger = CSVLogger(csv_log_path, append=True, separator=',')

callbacks = [early_stopping, model_checkpoint, tnsrboard, csv_logger]

history = model.fit(
    x_train,
    y_train,
    epochs=10,
    validation_split=0.2,
    batch_size=40,
    verbose=2,
    callbacks=callbacks
)
```

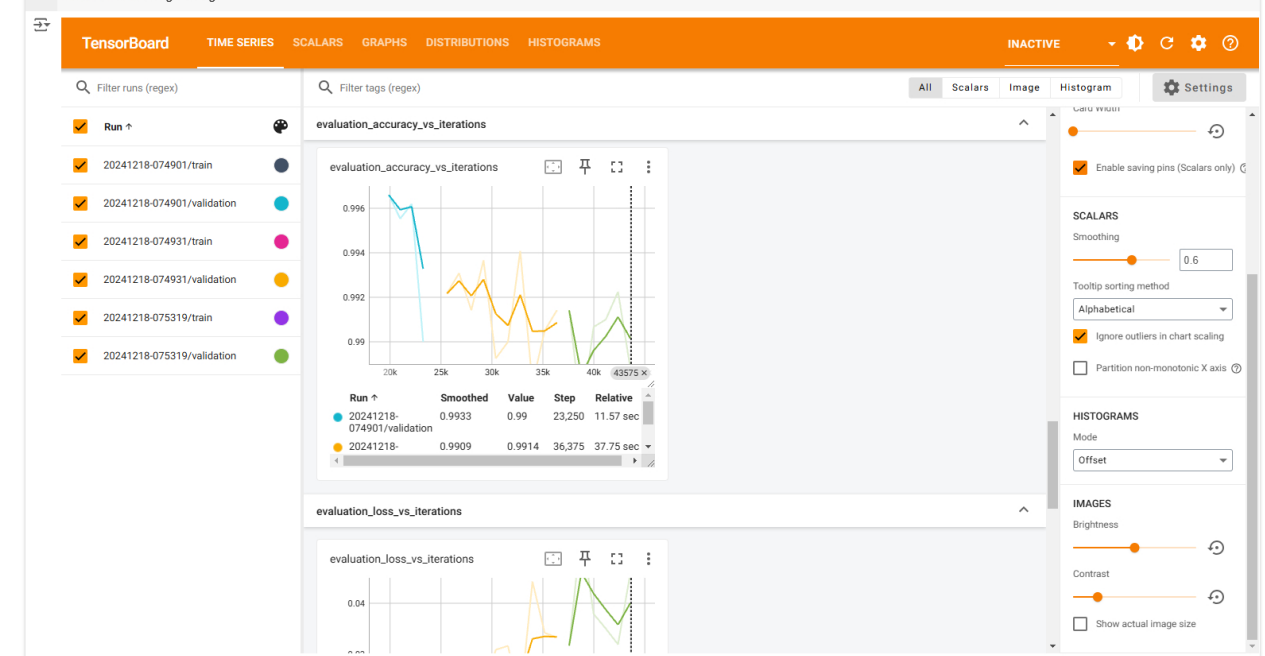
```
Epoch 1/10
1200/1200 - 3s - 2ms/step - accuracy: 0.9995 - loss: 0.0017 - val_accuracy: 0.9914 - val_loss: 0.0316
Epoch 2/10
1200/1200 - 6s - 5ms/step - accuracy: 0.9990 - loss: 0.0028 - val_accuracy: 0.9868 - val_loss: 0.0547
Epoch 3/10
1200/1200 - 3s - 3ms/step - accuracy: 0.9987 - loss: 0.0044 - val_accuracy: 0.9907 - val_loss: 0.0379
Epoch 4/10
1200/1200 - 5s - 4ms/step - accuracy: 0.9992 - loss: 0.0023 - val_accuracy: 0.9910 - val_loss: 0.0350
Epoch 5/10
1200/1200 - 3s - 3ms/step - accuracy: 0.9999 - loss: 8.2729e-04 - val_accuracy: 0.9923 - val_loss: 0.0319
Epoch 6/10
1200/1200 - 4s - 3ms/step - accuracy: 0.9979 - loss: 0.0067 - val_accuracy: 0.9888 - val_loss: 0.0459
```

```
[28] #The saved model can then be loaded and evaluated any time by calling the load_model() method
from keras.models import load_model
saved_model = load_model("/content/model.keras")

test_acc = saved_model.evaluate(x_test, y_test)
```

```
313/313 — 1s 2ms/step - accuracy: 0.9771 - loss: 0.1216
```

```
%load_ext tensorboard
%tensorboard --logdir logs
```



```
[ ] '''Regularizers :
from tensorflow.keras.regularizers import l2, l1
model.add(tf.keras.layers.Dense(128, activation='relu'))
model.add(tf.keras.layers.Dense(64, activation='relu'))
```

```
model.add(tf.keras.layers.Dense(4, activation='relu', kernel_regularizer=l1(l1=0.1)))
model.add(tf.keras.layers.Dense(4, activation='relu', kernel_regularizer=tensorflow.keras.regularizers.l1_l2(l1=0.1, l2=0.01)))
...
```

[Colab paid products](#) - [Cancel contracts here](#)

✓ 5s completed at 8:53 AM

